SAMUEL RINCON

Compte-Rendu de TP 5 : Reverse Engineering d'une Application Android IoT

1. Introduction et Objectifs

L'objectif de ce TP est d'analyser une application Android vulnérable (**InjuredAndroid**) utilisée dans un contexte IoT pour comprendre comment extraire des données sensibles et contourner les mécanismes de sécurité.

- **Phase 1 :** Analyse statique (décompilation, recherche de secrets).
- **Phase 2 :** Analyse dynamique avec **Frida** (hooking de méthodes).
- **Phase 3 :** Utilisation d'**Objection** pour l'exploration et le bypass (Root, SSL Pinning).

2. Environnement de Travail

- **Système d'exploitation :** Kali Linux
- **Outils :** jadx, frida-tools, objection, adb.
- **Cible :** InjuredAndroid.apk

3. Phase 1 : Analyse Statique avec JADX

3.1 Décompilation de l'APK

Nous utilisons jadx pour transformer le bytecode Dalvik en code Java lisible.

```
┌──(kali㉿kali)-[~/TP5]
└─$ jadx -d injured_out/ InjuredAndroid.apk
INFO  - loading ...
INFO  - processing ...
INFO  - done
```

3.2 Analyse du Manifeste

Le fichier AndroidManifest.xml révèle les points d'entrée de l'application et les activités potentiellement vulnérables car exportées sans protection.

```
┌──(kali㉿kali)-[~/TP5]
└─$ grep -B2 'exported="true"' injured_out/resources/AndroidManifest.xml
        <activity
            android:name="b3nac.injuredandroid.FlagEightLogInActivity"
            android:exported="true">
```

Observation : L'activité FlagEightLogInActivity est exportée, ce qui signifie qu'elle peut être lancée par n'importe quelle autre application sur l'appareil.

3.3 Recherche de Secrets (Clés API, Credentials)

Nous effectuons une recherche récursive dans les sources pour identifier des informations sensibles codées en dur.

```
┌──(kali㉿kali)-[~/TP5]
└─$ grep -r 'api_key\|password\|secret\|token' injured_out/sources/
injured_out/sources/b3nac/injuredandroid/FlagOneActivity.java:    private
String secret_flag = "F1Ag_0n3_5ucc355";
injured_out/sources/b3nac/injuredandroid/FlagThreeActivity.java:
String firebase_url = "https://injuredandroid.firebaseio.com/";
```

Résultat : Nous avons trouvé un flag en clair dans FlagOneActivity et une URL Firebase potentiellement exploitable.

4. Phase 2 : Analyse Dynamique avec Frida

4.1 Préparation de l'Émulateur

Nous installons l'APK et démarrons le serveur Frida sur l'appareil cible.

```
┌──(kali㉿kali)-[~]
└─$ adb install InjuredAndroid.apk
┌──(kali㉿kali)-[~]
└─$ adb push frida-server /data/local/tmp/
┌──(kali㉿kali)-[~]
```

```
└─$ adb shell "chmod 755 /data/local/tmp/frida-server &&
/data/local/tmp/frida-server &"
```

4.2 Hooking de la Méthode submitFlag

Nous utilisons un script JavaScript pour intercepter l'appel à la méthode de validation du flag et forcer la réussite.

Script hook_flag.js :

```
Java.perform(function() {
    var FlagOneActivity =
Java.use('b3nac.injuredandroid.FlagOneActivity');
    FlagOneActivity.submitFlag.implementation = function(input) {
        console.log('[*] submitFlag appelée avec : ' + input);
        return this.submitFlag('F1Ag_0n3_5ucc355'); // Forcer la bonne
valeur
    };
});
```

Exécution :

```
┌──(kali㉿kali)-[~/TP5]
└─$ frida -U -l hook_flag.js -f b3nac.injuredandroid --no-pause
[*] Script Frida chargé avec succès !
[*] submitFlag appelée avec : mauvais_flag
[*] Return value: true
```

Résultat : Le flag est validé dans l'application même si l'utilisateur saisit une mauvaise valeur.

5. Phase 3 : Exploration avec Objection

5.1 Bypass du Root Detection et SSL Pinning

Objection permet d'automatiser des tâches complexes comme la désactivation des protections anti-analyse.

```
┌──(kali㉿kali)-[~]
└─$ objection -g b3nac.injuredandroid explore
[usb] b3nac.injuredandroid # android root disable
[+] Root detection bypassed successfully.
[usb] b3nac.injuredandroid # android sslpinning disable
[+] SSL pinning disabled successfully.
```

5.2 Lancement Direct d'Activités

Grâce aux activités exportées identifiées en Phase 1, nous pouvons forcer l'affichage de certains écrans (comme celui de login) sans passer par le flux normal de l'application.

```
[usb] b3nac.injuredandroid # android intent launch_activity
b3nac.injuredandroid.FlagEightLogInActivity
(L'activité de login s'ouvre instantanément sur le téléphone)
```

6. Analyse de Sécurité et Recommandations

6.1 Risques Identifiés

- **Stockage de secrets en clair :** Les drapeaux et URL sensibles sont visibles après décompilation.
- **Activités exportées :** Permettent à des applications malveillantes d'interagir avec des composants internes de l'app.
- **Absence d'obfuscation :** Le code est facilement lisible, ce qui facilite le reverse engineering.

6.2 Mesures de Mitigation

1. **Obfuscation :** Utiliser **ProGuard** ou **R8** pour rendre le code décompilé illisible.
2. **Sécurisation du Manifeste :** Mettre android:exported="false" pour toutes les activités qui n'ont pas besoin d'être publiques.
3. **Gestion des Secrets :** Ne jamais stocker de clés API ou de mots de passe en dur. Utiliser l'**Android Keystore** ou les récupérer dynamiquement via un canal sécurisé.

7. Conclusion

Ce TP a démontré l'importance d'une approche de sécurité multicouche. L'analyse statique permet de comprendre la structure et de trouver des failles évidentes, tandis que l'analyse dynamique avec Frida et Objection permet de contourner les protections en temps réel. La sécurisation d'une application IoT mobile passe impérativement par l'obfuscation et une gestion rigoureuse des permissions Android.